

Does a Discovered Reinforcement Learning Rule Minimise Anything?

A gradient-field audit of DiscoRL, and what it does in the MDP whose soft-optimal policy is exactly a GFlowNet

Tianchi Chen

September 1, 2026

Abstract

DiscoRL replaces a hand-written reinforcement learning update with a meta-learned one, and the meta-parameters of its strongest rule, Disco103, are public. That makes a question askable that could not be asked of a rule nobody had written down: *is the discovered update the gradient of anything?* The question is not rhetorical. By the Poincaré lemma a vector field is a gradient exactly when its Jacobian is symmetric, so “which objective does this rule minimise” is well posed only if a single scalar, $\kappa = \|\text{sym } J\|_F / \|J\|_F$, equals one.

We measure it. Working in prediction space rather than parameter space, the non-conservative part of the field is isolated in the bootstrap operator $B = \partial \hat{p} / \partial u$ — precisely the term `stop_gradient` discards — and we estimate $\kappa(B)$ by paired Hutchinson probes through a single forward map. Two things make the number readable. First, an analytic reference: a *causal* bootstrap, whose target reads only the future, is strictly triangular, so $\text{tr}(B^2) = 0$ and $\kappa = 1/\sqrt{2}$ exactly, in any dimension. Second, an estimator validated before it is believed, on synthetic fields spanning $\kappa \in [0.025, 1]$ and again in the narrow band where the answer lands, and returning exactly 1.0000 for a genuine gradient.

Across six configurations — two environments, one of them from Disco103’s own meta-training family, and training lengths spanning a factor of sixteen — $\kappa(B) = 0.7084$ to 0.7094 , against a floor of 0.70711 . Disco103’s update is therefore not the gradient of any functional, and it carries no more circulation than causal bootstrapping forces. It is also, and this matters for how the first sentence should be read, barely non-conservative: the bootstrap term is 1.1% of the update field in Frobenius norm, so the antisymmetric share of the whole update is 0.77%. The obstruction is structural and small, and reporting either half alone misleads. Three probes of what the discovered predictions represent all return nothing, against a positive control that works. We report the traps that silently return the wrong answer, and the claims of our own that we withdrew.

That answer makes a second question askable. Tiapkin et al. prove that a GFlowNet is exactly the soft-optimal policy of an explicit MDP — interior rewards $\log P_B$, terminal reward $\log R$, entropy coefficient one — so “meta-learn a GFlowNet update rule” is a well-posed plan and not an analogy. We check the reduction on a hypergrid and it holds to 10^{-16} , including over fifty random backward policies. The MDP it produces then defeats Disco103: the rule stops at the source, 4.285 below the achievable return, while the same rule, network, observation and budget solve the same grid to within 0.552 once the per-step $\log P_B$ payments are removed. Potential shaping by a reward-free geometric potential recovers most of that, to a gap of 0.323, and provably does not move the target: the soft-optimal policy moves by 1.776×10^{-15} . Asking which tempered family the resulting sampler belongs to returns no verdict, for a reason worth reporting: the best-fit residual is predicted by how far a seed got from the achievable return

at $R^2 = 0.993$ and extrapolates to 0.0410 at zero gap, inside a measured ceiling of 0.0861. At the best-optimising seed the sampler is inside both candidate families at once, about five times colder than the coefficient that would make it a GFlowNet. What does separate rules is cheaper than a new architecture. Changing one scalar in the meta-objective, from the return to $V_1 = \log Z - D_{\text{KL}}(q\|P_B)$, moves a meta-learned local rule from D_{KL} 1.7035 to 0.0666 on held-out landscapes, a factor of 26 with architecture, inputs, inner horizon and seeds identical. At matched budget that is about nineteen times closer than detailed balance; given thirteen times more steps detailed balance is about six times closer than the rule. It is a result about speed, not about asymptotic accuracy.

1 Introduction

A reinforcement learning algorithm is usually an equation. DiscoRL [1] replaces the equation with a network: a meta-network reads a trajectory and emits targets, the agent regresses onto them, and the meta-network itself is trained by meta-gradient across a population of agents. The resulting rule, Disco103, matches or beats hand-designed algorithms on Atari, ProcGen and DMLab, and generalises to environments never seen during discovery. Its meta-parameters ship under an open licence.

That artefact makes a structural question concrete. Hand-designed rules come with a story about what they optimise — policy gradient ascends expected return, trajectory balance [5] descends an explicit residual. A discovered rule comes with no such story. It is natural to ask which objective it implicitly minimises, and a good deal of theory would follow if one could name it.

The question presupposes that an objective exists. It need not. Temporal difference learning famously is not the gradient of any function: the target is held fixed when the gradient is taken, and the discarded term is what a genuine gradient would have included [2, 3]. DiscoRL is built the same way — produce a target, regress onto it — so the presupposition has to be tested rather than assumed.

By the Poincaré lemma the test reduces to one scalar. On a simply connected domain a smooth field v is $-\nabla\Phi$ for some Φ if and only if its Jacobian is symmetric everywhere. Writing $\text{sym } M = (M + M^\top)/2$ and $\text{asym } M = (M - M^\top)/2$, and noting that these are orthogonal in the Frobenius inner product, define

$$\kappa(M) = \frac{\|\text{sym } M\|_F}{\|M\|_F}, \quad \kappa = 1 \iff M \text{ symmetric.} \quad (1)$$

This paper measures κ for Disco103.

Contributions.

1. An analytic reference scale. Any bootstrap whose target reads only the future is strictly triangular, hence has $\kappa = 1/\sqrt{2}$ exactly, independent of dimension and of every other detail (Proposition 1). Without it a measured κ is uninterpretable.
2. A measurement procedure that survives three traps, each of which returns a confidently wrong answer rather than an error (Section 3), with an estimator validated twice before use.
3. The measurement: $\kappa = 0.7084\text{--}0.7094$ over six configurations, including the environment family Disco103 was meta-trained on. The update is not a gradient field, and sits on the causal floor.

4. Three negative probes of what the discovered predictions carry, with a positive control that works, bounding how strongly the negatives may be read.
5. A withdrawal section. Six interpretations we published in earlier drafts of this work were wrong, and we say which and why. One of them is a headline result of the second half, retracted by a control we ran ourselves.
6. A verified reduction and a negative result on top of it: the GFlowNet-to-RL correspondence holds numerically to 10^{-16} (Section 6), and the MDP it produces defeats the discovered rule outright, with a wiring control that isolates the MDP as the cause (Section 7).
7. A measured scale for “large residual” in place of a picked threshold, and a pipeline check that closes the one link the two estimator validations were blind to (Section 9). Against that scale the family question turns out to be confounded by optimisation, and we withdraw our own answer to it (Section 10).
8. A positive result of the same size as the negative ones: one scalar in the meta-objective, chosen by the reduction rather than by search, is worth a factor of 26 in held-out distribution accuracy (Section 11).

2 The field and where its asymmetry lives

DiscoRL’s per-step agent loss is a sum of categorical Kullback–Leibler terms between meta-network targets and agent predictions,

$$\mathcal{L}_\theta = c_\pi D_{\text{KL}}(\hat{\pi} \parallel \pi_\theta) + c_y D_{\text{KL}}(\hat{y} \parallel y_\theta) + c_z D_{\text{KL}}(\hat{z} \parallel z_\theta(\cdot, a)) + \cdots \quad (2)$$

with y and z carrying no predefined semantics. In the released implementation $\hat{\pi}, \hat{y}, \hat{z}$ are wrapped in a stop-gradient when the agent update is taken, so the update is a *semi*-gradient.

We work in prediction space rather than parameter space. This is legitimate and also convenient. If a field is a gradient in prediction space it pulls back to a gradient in parameter space, since $-(\partial u / \partial \theta)^\top \nabla_u \Phi = -\nabla_\theta \Phi(u(\theta))$, so the same question is being asked; and prediction space needs only *first* derivatives of the target map, which matters for reasons given in Section 3.

Write u for the prediction logits and $p = \text{softmax}(u)$. The gradient of a categorical KL with respect to the prediction logits, target held fixed, is the familiar difference of probabilities, so the field is

$$v(u) = \hat{p}(u) - p(u), \quad J = \underbrace{\frac{\partial \hat{p}}{\partial u}}_{=:B} - \underbrace{\frac{\partial p}{\partial u}}_{=: \text{diag}(p) - pp^\top}. \quad (3)$$

The second term is symmetric, always. Every departure from a gradient field therefore lives in B , the dependence of the target on the prediction — which is exactly the term the stop-gradient discards. The measurement is $\kappa(B)$.

2.1 An analytic floor

Proposition 1. *For any square M , $\|M \pm M^\top\|_F^2 = 2\|M\|_F^2 \pm 2\text{tr}(M^2)$, hence*

$$\kappa(M)^2 = \frac{1}{2} + \frac{\text{tr}(M^2)}{2\|M\|_F^2}. \quad (4)$$

If M is strictly triangular in some ordering then M^2 is strictly triangular, so $\text{tr}(M^2) = 0$ and $\kappa(M) = 1/\sqrt{2}$ exactly.

Proof. $\langle M, M^\top \rangle_F = \text{tr}(M^\top M^\top) = \text{tr}((M^2)^\top) = \text{tr}(M^2)$, and expanding $\|M \pm M^\top\|_F^2$ gives the identity. Equation (4) follows from $\kappa^2 = \|M + M^\top\|^2 / (\|M + M^\top\|^2 + \|M - M^\top\|^2)$. Strict triangularity gives a nilpotent M with zero diagonal, so $\text{tr}(M^2) = \sum_{ij} M_{ij}M_{ji} = 0$. $\square$

Remark. A *causal* bootstrap — one whose target at index t depends only on indices after t — is strictly triangular in the time ordering. So $\kappa = 1/\sqrt{2} \approx 0.70711$ is the structural floor imposed by the arrow of time in credit assignment, and it is dimension-free. Tabular TD is the canonical instance: its target map is $\hat{V} = r + \gamma PV$, so $B = \gamma P$ and on any acyclic graph $\kappa = 1/\sqrt{2}$.

The floor is not forced for DiscoRL. Its meta-network receives its own y at index t as an input — the released input specification pairs y_t with y_{t+1} — so B has a nonzero diagonal block and $\text{tr}(B^2) \neq 0$ in general. Whether Disco103 nonetheless sits on the floor is an empirical question, and by (4) the excess over the floor is measurable directly as $\text{tr}(B^2)/\|B\|_F^2 = 2\kappa^2 - 1$.

2.2 Estimator

Frobenius norms come from paired Hutchinson probes: for $w \sim \mathcal{N}(0, I)$, $\|M\|_F^2 = \mathbb{E}_w \|Mw\|^2$, with Bw from a Jacobian-vector product and $B^\top w$ from a vector-Jacobian product *on the same* w . Pairing matters: the two norms are strongly correlated, so their ratio is far better conditioned than either term. Standard errors come from a block jackknife over probes.

Validation, twice, before use. On synthetic fields $g(x) = (S + cA)x$ with S symmetric and A antisymmetric, κ is known in closed form; sweeping c walks the estimator across $\kappa \in [0.025, 1]$ with worst absolute error 0.0036, and it returns *exactly* 1.0000 at $c = 0$. Since the answer lands near $1/\sqrt{2}$, we calibrate again there: strictly triangular B_0 plus a controlled diagonal block, where (4) gives the exact value, with worst error 0.0041. That 0.0041 is the resolution with which any excess over the floor can be claimed, and we quote it whenever we compare against the floor.

3 Three traps

Each of the following returns a plausible number rather than an error, and each was hit before it was fixed.

Second-order stop-gradient. `jax.lax.stop_gradient` has zero derivative at *all* orders. Differentiating the released `backprop=False` path a second time therefore erases B — the very object under measurement — and reports a perfect gradient field. Our first attempt returned $\beta = 0.0000$ from an autodiff path for exactly this reason. The cure here is structural: B is obtained as a *first* derivative of the target map, so no stop-gradient is ever on the path.

Querying an untrained agent. Disco103 was meta-learned against competent agents. Presented with a freshly initialised one — near-uniform policy, near-uniform 600-way predictions — its outputs collapse towards a constant and every sensitivity reads as zero for reasons unrelated to conservativity. The agent must first be trained *by Disco103 itself*.

Silent out-of-bounds. JAX drops out-of-bounds scatter indices silently. A control that perturbs index $t + 3$ while reading index t becomes a no-op at the end of a rollout, and the resulting 0/0 surfaces as `nan` rather than as an error. Relatedly, an ε placed outside rather than inside the square

Environment	Training steps	$\kappa(B)$	$\text{tr}(B^2)/\ B\ _F^2$	$\kappa - 1/\sqrt{2}$
hypergrid	100	0.7084 ± 0.0002	+0.00358	+0.0013
hypergrid	400	0.7086 ± 0.0007	+0.00435	+0.0015
hypergrid	1600	0.7090 ± 0.0009	+0.00550	+0.0019
Catch	100	0.7086 ± 0.0003	+0.00414	+0.0015
Catch	400	0.7088 ± 0.0004	+0.00477	+0.0017
Catch	1600	0.7094 ± 0.0008	+0.00648	+0.0023
causal floor $1/\sqrt{2}$		0.70711	0	—
estimator resolution in this band		0.0041		

Table 1: κ of Disco103’s bootstrap operator. Dimension 86,616, 48 paired probes, errors by block jackknife. Every excess over the floor is smaller than the estimator’s demonstrated resolution in the same band.

root in an unrolled Adam gives an infinite derivative at zero second moment — which masked-out logits always have — turning the outer gradient into **nan** while the forward pass stays finite.

4 Result

We train an agent with Disco103 on a given environment, freeze a rollout, and measure $\kappa(B)$ on the joint prediction vector of policy logits, y logits, and z logits at the action taken; the dimension is 86,616. Two environments are used. The first is a hypergrid, a DAG in which several action sequences reach the same state. The second is Catch, which belongs to the family Disco103 was actually meta-trained against and therefore answers the obvious objection that the first is an out-of-distribution query. Training length is swept over a factor of sixteen.

Table 1 gives the result. Two readings, both of which matter.

$\kappa \neq 1$: the update is not a gradient field. The estimator returns exactly 1.0000 for a genuine gradient, and the frozen-target control on the same network — the same loss with the target held constant, which is an honest scalar objective — also returns 1.0000. Against that, 0.7088 is not a near miss. Disco103’s update is not $-\nabla\Phi$ for any Φ , so *which objective does DiscoRL minimise* is malformed rather than open.

$\kappa \approx 1/\sqrt{2}$: no more circulation than causality forces. Every excess over the floor lies between +0.0013 and +0.0023, below the 0.0041 resolution demonstrated in that band. The non-conservativity is therefore not something meta-learning discovered; it is the generic price of a causal bootstrap. The excess is nonzero and increases monotonically with training length in both environments, consistent with the diagonal block that the paired (y_t, y_{t+1}) input creates, but it remains within resolution throughout.

How much of the update is this? $\kappa(B)$ is a property of the bootstrap term, not of the update. The field the agent follows is $v = \hat{p} - p$, so $J = B - D$ with D symmetric, and D is much the larger of the two. Measuring all three operators on the same probes gives $\kappa(D) = 1.0000$, as it

Readout	Target	R^2	Spearman	Role
q head	V_π	0.559	+0.844	positive control
$\varphi(z)$	V_π	0.478	-0.722	
$\varphi(y)$	V_π	0.125	-0.377	
$\varphi(y)$	$\log F$	0.018	-0.080	flow hypothesis
$\varphi(y)$	$\mathcal{H}_{\text{future}}$	0.002	—	entropy hypothesis

Table 2: State-conditional means over 64/64 visited states, 40 rollouts, exact targets by dynamic programming. The control works; the y channel tracks nothing.

must; $\|B\|_F/\|J\|_F = 0.0110$; and therefore an antisymmetric share of the whole update of

$$\frac{\|\text{asym } J\|_F}{\|J\|_F} = \sqrt{1 - \kappa(B)^2} \frac{\|B\|_F}{\|J\|_F} = 0.77\%. \quad (5)$$

Both statements are true and they are not the same statement. The bootstrap term is half circulation; the bootstrap term is one percent of the update. A reader who takes $\kappa(B) = 0.709$ to mean “the update is half non-conservative” is wrong by two orders of magnitude, and we would rather supply the second number than rely on the first being read carefully.

What survives, and what deflates. The qualitative claim is untouched: 0.77% is not zero, so no potential exists, no Lyapunov function is guaranteed, and the search for an implicit objective is a search for something that is not there. What deflates is the expected magnitude of any consequence. A field within one percent of conservative, with a strongly contracting symmetric part, is a small perturbation of a gradient flow, and one should predict a correspondingly small behavioural effect. We regard this as the more useful pair of facts: the diagnosis is structurally sound *and* quantitatively bounded. Reporting only $\kappa(B)$ would have made a one-percent effect sound like a fifty-percent one.

On the Hodge projection. By orthogonality of sym and asym, the projection onto the conservative sector retains $\kappa^2 \approx 0.502$ of the *bootstrap operator’s* Frobenius mass, and 99.994% of the update field’s. Neither number has been connected to sample efficiency or solution quality, which is work we have not done.

5 What do the discovered predictions represent?

DiscoRL’s own analysis reports that the learned predictions track future policy entropy and upcoming large-reward events. On a hypergrid the natural candidate state functions are computable exactly by dynamic programming, so the claim can be tested rather than paraphrased. We use Disco103’s *own* scalar readout of its 600-way prediction — the `y_net` and `z_net` multilayer perceptrons whose weights ship with the released parameters — so the probe has zero free parameters. The categorical q head, a value head by construction, serves as positive control.

Table 2 reports the outcome. The control passes cleanly, so the bucketing, the training and the dynamic programming all work; against that baseline the y channel tracks neither a flow, nor a value, nor future policy entropy. A joint regression does not rescue it: adding $\varphi(y)$ to $\varphi(z)$ buys 0.010 of R^2 on $\log F$ but 0.060 on the pure value control, so it behaves as a generic weak regressor rather than as the complementary half of a free-energy decomposition.

	Check	Result
R1	$\max_s V^*(s) - \log F(s) $	4.753×10^{-16}
R2	$ V^*(s_0) - \log Z $	0 exactly
R3	$\max_x p(x) - R(x)/Z $	2.776×10^{-17}
R4	same, over 50 random backward policies	worst 1.665×10^{-16}

Table 3: The reduction checked numerically on a hypergrid. R4 is the statement that the backward policy is a gauge freedom, exercised rather than argued.

How strongly may these nulls be read? A potential must be a state function, and the agent’s torso is recurrent, so we decompose each readout’s variance. $\varphi(y)$ is a *pure* state function, $\eta^2 = 1.0000$ — the bucketing was never washing it out — but its dynamic range is $68\times$ smaller than $\varphi(z)$ ’s. The honest scope is therefore: in this regime the y channel carries a deterministic but vanishingly small state signal that matches no candidate potential. That is weaker than “ y can never be a flow”.

A sensitivity that is real. A separate probe shows the y -target does respond to the log-probability of the action taken, at $139\times$ the random-initialisation null, and the response is local: moving the perturbation three steps later reduces it by $23\times$. The y channel is not inert as a map; it is unidentified as a function.

6 The reduction, verified

Everything from here concerns a second question, and the first answer is what makes it worth asking. If *which objective does this rule minimise* is malformed, one can still ask what a discovered rule does in a setting where the objective is fixed in advance and known exactly.

Tiapkin et al. [7] supply that setting. Their Theorem 1: for a DAG with terminal set $\mathcal{X}$, a fixed backward policy P_B and a reward R , build the MDP whose interior transitions pay $\log P_B(s | s')$ and whose termination pays $\log R(x)$. At entropy coefficient *exactly* 1 the soft-optimal policy is the GFlowNet forward policy and $V^*(s) = \log F(s)$. Their Proposition 1 gives the entropy-regularised return at the source in closed form, $V_1^\pi(s_0) = \log Z - D_{\text{KL}}(q^\pi \| P_B)$, with supremum $\log Z$.

We derive none of this. We check it, on a hypergrid, against an exact dynamic program.

Table 3 is the result. The coefficient is not a free parameter, and Remark 3 of the same paper says what moving it costs. Pricing that remark here, as the KL of the terminal distribution to R/Z , gives $\lambda = 0.9 \rightarrow 0.0367$, $1.0 \rightarrow -6.5 \times 10^{-17}$, $1.1 \rightarrow 0.0386$, $2.0 \rightarrow 1.134$ and $4.0 \rightarrow 1.782$. The reduction is exact at one and at no other coefficient, and the price of being wrong about it grows quickly.

This settles a question of well-posedness rather than of performance. Because the correspondence is exact, “meta-learn a GFlowNet update rule” names a concrete MDP with a concrete entropy coefficient, and a rule can be dropped into it and scored. The rest of the paper does that.

7 The MDP the reduction produces

The first thing to record is that Disco103 has no entropy coefficient to set. Its `hyper_params` are `pi_cost`, `y_cost`, `z_cost`, `value_cost`, `aux_policy_cost`, `target_params_coeff`, `value_fn_td_`

Arm	Return	Gap to best	Residual
disco, unshaped reduction	−2.000	+4.285	6.14
disco, same grid, log P_B payments removed	2.879	+0.552	(control)
disco, shaped reduction	1.961	+0.323	0.649

Table 4: Three arms on the same grid, three seeds. The middle row is a wiring control and carries no temperature meaning, so no residual is quoted for it.

`lambda` and `discount`. The actor-critic in the same library carries an `entropy_cost` of 0.2; the discovered rule carries no such knob. This is verified in source, not read off the paper.

The environment is a 6×6 grid with three smooth reward modes, log R spanning 5.43. The best undiscounted return available is 2.285 with the log P_B payments in place and 3.431 without them. Stopping at the source pays −2.000.

The failure is the MDP, not the wiring. The unshaped number is identical to three decimals across all three seeds, which is already a hint that nothing stochastic is happening. The middle row of Table 4 settles it: the same rule, the same network, the same observation and the same budget solve the same grid to a gap of 0.552 once the per-step payments are removed. That row is a control on the harness and nothing else. It is not an arm at some other temperature, and its residual would not mean anything, so we do not report one.

Mechanism. Each interior step pays about −0.69. Stopping immediately therefore returns −2.000, which is a genuine improvement over the initial policy’s −2.281, and escaping that basin requires a specific five-step path. The rule takes the improvement and stays. This is Remark 4 of [7] — soft actor-critic with an adaptive coefficient converging to a biased final policy — reproduced for a discovered rule, which has no coefficient to adapt in the first place.

8 Shaping

The basin is an optimisation obstacle, and potential shaping is the standard instrument for one. Let Φ be the soft value at coefficient 1 of the same DAG with the reward flattened away. This matters and is the whole reason the section exists: Φ is pure graph geometry and never reads R , so it cannot be the answer smuggled in as a hint.

For the shaped reward $r' = r + \Phi(s') - \Phi(s)$,

$$\lambda \log \sum_a \exp((r' + V'(s'))/\lambda) = V(s) - \Phi(s), \quad (6)$$

$$\pi' \propto \exp((r + V(s'))/\lambda) = \pi. \quad (7)$$

The value function shifts by Φ and the soft-optimal policy does not move.

We checked this pointwise rather than resting on the algebra. The maximum change in the soft-optimal policy over $\lambda \in \{0.1, 0.5, 1, 2, 5\}$ is 1.776×10^{-15} . So the third row of Table 4 is the same target reached from a better-conditioned starting point, and the gap it closes, 4.285 down to 0.323, is entirely optimisation.

Check	Result
member at $\lambda = 1$ equals R/Z	1.665×10^{-16}
I1, fed exact members, no sampling	all 7 coefficients to 4 decimals, residual 10^{-16}
I2, sample $\rightarrow$ bucket $\rightarrow$ DP $\rightarrow$ fit	recovers to 0.5%
bucketing consistency	10^{-17}

Table 5: Validation of the readout before it is believed.

Temperature error	Nats
10%	0.0151
25%	0.0861
50%	0.2986
100%	0.9345
family diameter, $\lambda = 0.05$ to 6.0	24.55

Table 6: The cost of being wrong about the coefficient. A residual below 0.0861 cannot be distinguished from a temperature error of a quarter.

9 The instrument

Asking whether the resulting sampler is a tempered one requires an instrument, and the instrument has to be trusted before its readings are. The soft family is computed exactly by value iteration; an agent is bucketed into a state-conditional policy; the coefficient is fitted by golden section against exactly recomputed members. The fit returns a coefficient *and* the residual divergence that no coefficient removes, and it is the second number that was supposed to decide family membership.

The bucketing check carries no sampling noise. On a DAG whose trajectories all begin at the source,

$$\text{reach}(c) = \sum_p \frac{n(p)}{N} \frac{n(p \rightarrow c)}{n(p)} = \frac{n(c)}{N}, \quad (8)$$

so the forward dynamic program run under the bucketed policy reproduces the empirical visit frequencies identically. The 10^{-17} in Table 5 is therefore not a small statistical fluctuation that happened to come out small; any departure from zero would measure dependence on history alone.

The link both validations were blind to. I1 and I2 share a gap. The agents act in the `disco_rl` environment and their rollouts are handed over wholesale, including rollout-boundary partial episodes, which the exact sampler used in I1 and I2 never produces. `temperature_pipeline_check.py` closes it by driving the exact soft-optimal policy through that same environment path and asking the readout what it sees: $\lambda_{\text{eff}} = 0.995$ at residual 0.0062. The pipeline is sound, so the residuals the arms show are theirs.

What counts as large, measured rather than picked. A residual is only interpretable against the most a temperature error can cost anywhere in this family, so we measured that.

`temperature.py` originally declared an agent outside the family when the residual exceeded 0.4. That number was picked and not measured. It has been replaced by the 25% figure, 0.0861, and

Seed	Adm.	Return	Gap	Soft λ	Soft res.	Fixed-pt res.	Ceiling
0	yes	2.247	0.038	0.1857	0.0649	0.0626	both
1	yes	2.219	0.066	0.2541	0.2091	0.1860	neither
2	yes	1.347	0.937	0.6217	1.4234	0.7091	neither
3	no	2.102	0.182	0.0300	0.0413	0.0413	both
4	yes	1.903	0.382	0.3548	0.5470	0.4164	neither
5	yes	2.223	0.061	0.2185	0.1226	0.1143	neither

Table 7: The rows behind the means. Best achievable return in this MDP is 2.2847, and the gap is the shortfall against it. The last column records whether the seed sits inside the 0.0861 ceiling for each family.

the replacement is a withdrawal (Section 12). It is also what makes the next section’s arithmetic readable, since without a scale there is nothing to compare an intercept against.

10 The misfit is the optimisation gap

The ladder was never a calibration. `temperature.py` calibrated its readout against actor-critic arms at `entropy_cost` $c \in \{0, 0.3, 1, 3\}$, and an earlier version of that file described the arm at $c = 1$ as “an arm whose coefficient is exactly one by construction”. That is false, and seeing it requires no training run at all: the exact fixed point of that update rule is $\pi \propto \exp(Q_\pi/c)$ with the *ordinary* Q_π , which is a different one-parameter family. It coincides with the soft family below $c \approx 0.5$ and leaves it above, reaching residual 3.02 at $c = 3$.

The fixed point predicts arms it was never fitted to. At $c = 3$ it gives residual 3.021 against the measured 3.009, and λ_{eff} 1.636 against the measured 1.639. At $c = 1$ it sits at $\lambda_{\text{eff}} = 0.836$ with residual 0.685, eight times the ceiling of Table 6. No rung of the ladder pinned the absolute scale; the ladder ranks, it does not calibrate.

Computing the c -to- λ map by dynamic programming repairs the reading. The map is monotone and invertible, and through it the shaped arm’s aggregate 0.362 is an equivalent entropy coefficient of 0.418, against the 1.181 that would place an arm at $\lambda = 1$. The map is what allows a coefficient in one family to be compared with a coefficient in the other at all. The aggregate it converts, however, does not survive what follows.

A verdict we withdraw. The soft family answers “is it a GFlowNet”. The fixed-point family above answers “is it an entropy-bonus actor-critic”. We fitted both to the shaped arm and reported the means over final-step admissible rows: soft 0.6482 and fixed-point 0.3720 at three seeds, soft 0.4734 and fixed-point 0.2977 at six. All four are above the 0.0861 ceiling, and we read that as *neither family*. The reading is withdrawn. The per-seed rows behind those means run from 0.041 to 1.423, and that range is not noise.

Regressing residual on gap over the five admissible seeds,

$$\text{soft residual} = 1.457 \times \text{gap} + 0.0410, \quad \text{Pearson } + 0.996, R^2 = 0.993, \quad (9)$$

$$\text{fixed-point residual} = 0.680 \times \text{gap} + 0.0958, \quad \text{Pearson } + 0.982, R^2 = 0.963. \quad (10)$$

Leave-one-out on the soft fit keeps Pearson between 0.977 and 0.999 and the intercept between 0.0089 and 0.0599 nats, inside the ceiling in all six fits, so no single seed carries it.

Value head	λ_{eff}	Residual
categorical, <code>mav</code> = 8	1.0145	1.531
scalar	1.0148	1.553

Table 8: Value resolution is excluded as the source of that arm’s residual.

A family misfit would not care how well the agent optimised. This one is predicted by it at R^2 0.993 and vanishes into the measured ceiling as the gap goes to zero. The residual we were reading was a measurement of the difficulty Section 7 established, not of family membership, and the verdict is withdrawn (Section 12).

What the same rows do support. Two statements, both weaker than the one they replace.

First, once the discovered rule actually optimises this MDP, its terminal distribution sits within a 25% temperature error of the soft family, at a coefficient near 0.19 to 0.22. That is about five times colder than the coefficient that would make it a GFlowNet. We write it that way rather than as a number, because the spread across seeds at comparable gaps is 0.186 to 0.254.

Second, this MDP does not separate the two families. At the best-optimising admissible seed the two residuals differ by 0.0023 nats, against a ceiling of 0.0861. We say that rather than claiming either family wins.

The one residual that is not a temperature error. One reading in this neighbourhood does survive, and it is about the ladder rather than about the discovered rule. The arm `ladder acc=1.0` reads $\lambda_{\text{eff}} = 1.014$, the right location, while leaving residual 1.564; a 25% temperature error costs at most 0.0861 nats, so that is not a temperature error. Swapping the value head shows it is not value resolution either.

11 One scalar in the meta-objective

The sections above are diagnostic, and they share a shape: a discovered rule is scored against a target it was never given. Proposition 1 of [7] says the target is available cheaply, so the last experiment gives it to the meta-learner and changes nothing else.

The identity, checked first. In double precision, on the shaped rewards, $V_1 + \Phi(s_0) = \log Z$ to 0.00e+00, and the policy attaining it has divergence 5×10^{-16} . The entropy-regularised return in this MDP *is* distribution accuracy, at the price of a scalar computed from trajectories. Nothing about the rule needs to know that.

The rule is local. Per transition it reads the immediate reward, its own auxiliary prediction at both ends, the current logit and log-probability, and whether the action terminates. It never sees R/Z , $\log Z$ or the visit distribution. Everything downstream of the policy is closed form — the grid is a layered DAG, so occupancy is $e_0^\top(I - T)^{-1}$ — which removes sampling noise from the meta-gradient. What is reported is therefore the infinite-sample limit of the same rule. Meta-training uses 16 random landscapes and the readout is on 16 disjoint ones.

Meta-objective	D_{KL} at 30 steps	D_{KL} at 120 steps
the return	1.7035 ± 0.523	1.6650 ± 0.508
$V_1 = \log Z - D_{\text{KL}}$	0.0666 ± 0.091	0.1043 ± 0.114

Table 9: Held-out landscapes, three seeds. Architecture, inputs, inner horizon and seeds are identical between the rows; only the scalar the meta-gradient ascends differs.

	D_{KL}
DB, 30 Adam steps (budget matched to the rule)	1.2570
learned rule, 30 steps	0.0666
DB, 400 Adam steps	0.0102

Table 10: Matched budget and unmatched budget, on the same landscapes and under the same weighting.

Two controls. First, each arm chooses its own step sizes, scored by *its own* meta-objective on training landscapes and never by divergence to the target. Second, each rule must win on the scalar it was trained on. It does, at the trained horizon and four times beyond: on the return the return-trained rule scores -0.341 against -1.327 , and on V_1 the V_1 -trained rule scores $+2.231$ against -0.298 . An earlier run of this comparison showed an $88\times$ separation. The second control voided it — the return-trained rule was losing on return itself, so that run compared a trained rule against an undertrained one — and that is a withdrawal (Section 12).

Table 9 is a factor of 26 from one scalar. The seed spread, 0.091 on a mean of 0.067, is not small, and we would not report a factor of two on that evidence.

Against the hand-designed loss. Detailed balance at its best weighting is the reference point.

At matched budget the learned rule is about 19 times closer. Given 13 times more steps, detailed balance is about 6 times closer than the rule. This is a result about speed and not about asymptotic accuracy, and it must be stated that way; the learned rule does not beat detailed balance in general and we make no such claim.

A property of the baseline, found on the way. The same detailed-balance loss lands anywhere from 0.010 to 2.535 as its weighting goes from full support to purely on-policy. Weighted purely on-policy it drives its *own* residual to 10^{-4} while sitting at D_{KL} 0.92, because states the policy abandons are never corrected. Rule and baseline are given the same weighting throughout, so the comparison above is about the update and not about exploration.

12 Withdrawals

An earlier draft of this work reported two interpretations that we now retract. Both arose from the same failure: reading a number that has a simple structural explanation as evidence for a more elaborate one.

First, we reported the ratio of the y -target’s sensitivity to log-policy against its sensitivity to the successor prediction, $|\beta/\alpha| = 0.32$, as “a third of the way to detailed balance”. That reading

presumes $\varphi(y)$ is a log-flow quantity. The calibration in Table 2 refutes the premise, so the reading is withdrawn; the sensitivity measurement itself stands.

Second, we read $\kappa^2 \approx 0.5$ as evidence that the update implements a saddle-point, primal–dual scheme, on the grounds that such schemes have an irreducibly antisymmetric coupling block. But Proposition 1 already accounts for the same number with no saddle structure whatsoever, and it was derived first. The simpler explanation suffices; the saddle reading is withdrawn.

Four further retractions belong to the second half of this work, and the last is the largest.

Third, the residual threshold 0.4 in `temperature.py`, above which an agent was declared outside the soft family, was picked and not measured. It is replaced by 0.0861, the cost of a 25% temperature error, which is measured (Table 6).

Fourth, `temperature.py` treated the actor-critic `entropy_cost` as the soft family’s coefficient, and an earlier version of that file called the arm at $c = 1$ an arm whose coefficient is one by construction. It is not: the fixed point is $\pi \propto \exp(Q_\pi/c)$ with the ordinary Q_π , a different family (Section 10). The ladder ranks, it does not calibrate, and the calibration reading is withdrawn.

Fifth, we reported an $88\times$ separation between the two meta-objectives. Its own control voided it: the return-trained rule was losing on return, so the comparison was between a trained rule and an undertrained one. The separation we stand behind is the factor of 26 in Table 9.

Sixth, we reported a verdict of *neither family* for the shaped arm, from the means of the family comparison. The residual it rested on is predicted by the optimisation gap at R^2 0.993 and extrapolates to 0.0410 nats at zero gap, inside the measured ceiling, so it was never evidence about family membership (Section 10). The verdict is withdrawn. The reproducer is `research/temperature_gap_confound.py`. What replaces it is weaker and is stated as such: at the best-optimising seed the arm is inside both families, and this MDP does not separate them.

The withdrawals of the earlier draft stand as well: the gauge theorem conceded to Brunswic et al., the detailed-balance reading of $|\beta/\alpha|$, the saddle reading of κ^2 , and the Hatano–Sasa equality demoted to a structural isomorphism.

13 Related work, and what is not ours

The gradient-field question for bootstrapped updates is old. Residual-gradient methods differentiate through the target and are genuine gradient methods, at the cost of a double-sampling bias in stochastic environments and a different, often worse, fixed point under function approximation [2]; the gradient-TD family restores a true objective by introducing an auxiliary variable [3], and the convex-duality view organises this [4]. What is new here is not the question but its application to a rule that was discovered rather than designed, together with an analytic reference scale that makes the answer readable.

We also record what is *not* ours. In the course of this work we derived a structural result about GFlowNet flow spaces — that the set of R -flows is an affine set directed by the cycle space of the state graph, so the backward policy is a gauge freedom that cannot move the terminal distribution. It is published. Brunswic et al. [6] prove $\mathcal{F}_R = F + H^1(G)$ and attribute a stronger form to Kalpazidou; that every backward policy induces the same terminal distribution is standard and stated in at least six papers, most crisply in trajectory balance [5], which already names cycles in the underlying undirected graph as the source of the multiplicity; and learning the backward policy for performance with correctness preserved by construction has been done with at least five distinct objectives. Our addition was the dimension $|E| - |V| + 1$ and its identification with the parameter count of the backward policy, which is the first Betti number and follows immediately. We claim none of it.

We note also that the method we had been using to judge novelty — counting arXiv abstract matches — failed on exactly this case, because the pre-empting paper writes $H^1(G)$ in cohomological notation and never uses the phrase “cycle space”.

14 Limitations

The measurement is of one rule, Disco103, on two environments. Catch answers the distribution objection but neither environment is ProcGen or DMLab. κ is computed on a frozen rollout at a single point in parameter space, and while it is stable across a sixteen-fold range of training length, it is a local quantity. The probe restricts to the prediction channels the agent loss touches and holds the Polyak-delayed target network fixed, which is faithful within one update but suppresses one dependency. Batch elements couple at $O(1/B)$ through the advantage and TD normalisers. Finally, both conservative-mass statements — $\kappa(B)^2 \approx 0.5$ for the bootstrap term and a 0.77% antisymmetric share for the update — are operator-level facts with no demonstrated behavioural consequence. The natural next step is the counterfactual: let the gradient flow through the target as well, which makes the update conservative by construction, and measure what performance changes. The magnitudes above predict that little will.

The second half of the paper carries its own bounds. It is a 6×6 grid with a tabular policy and exact expectations; Section 11 is an infinite-sample limit and not a sampled algorithm. That section compares against detailed balance only, and loses to it asymptotically. Its seed spread is large relative to its mean. The positive reading in Section 10 rests on the single best-optimising admissible seed plus an extrapolation to zero gap; it is not three independent seeds at zero gap. And all of it is one reward family on one graph topology.

15 Reproducibility

Every number in this paper is produced by a script, and the scripts are in the accompanying repository. `kappa.py` performs the estimator validation and the measurement; `calibrate.py` and `thermo.py` perform the prediction probes and the variance decomposition; `disco_probe.py` performs the sensitivity measurement; `verify_disco_source.py` encodes the claims we make about the released implementation as executable assertions pinned to commit 9059a29f, re-run weekly, so that a claim about someone else’s code cannot rot silently. All exact quantities are computed by dynamic programming over the state graph rather than sampled.

The second half adds nine. `tiapkin.py` checks R1–R4 of the reduction and prices Remark 3. `temperature.py` builds the reduction MDP, runs the three arms of Table 4 and fits the soft-family coefficient. `temperature_pipeline_check.py` drives the exact soft-optimal policy through the agents’ own environment path, returning $\lambda_{\text{eff}} = 0.995$ at residual 0.0062, which is the evidence that the readout’s residuals belong to the arms. `temperature_residual_scale.py` measures what a temperature error costs and is where 0.0861 comes from. `temperature_rule_fixedpoint.py` computes the actor-critic rule’s exact fixed point and the c -to- λ map, and is what shows the ladder never calibrated. `temperature_family_compare.py` fits both families to the same arm at three and six seeds. `temperature_gap_confound.py` rereads its per-seed rows against the optimisation gap and is what withdrew our family verdict. `temperature_critic_ablation.py` swaps the value head and excludes value resolution. `metagfn.py` meta-learns the local update rule under each scalar and runs the two controls and the detailed-balance comparison.

One provenance note, because it affects how these can be checked. `temperature_`

`pipeline_check.py`, `temperature_rule_fixedpoint.py`, `temperature_residual_scale.py`, `temperature_critic_ablation.py` and `temperature_family_compare.py` were written in a concurrent session and were swept into commit `e6db278` by a `git add -A` whose message does not describe them. A `git note` on that commit records this. Sections 9 and 10 rest on them.

References

- [1] J. Oh, G. Farquhar, I. Kemaev, D. A. Calian, M. Hessel, L. Zintgraf, S. Singh, H. van Hasselt, D. Silver. Discovering state-of-the-art reinforcement learning algorithms. *Nature*, 2025. <https://doi.org/10.1038/s41586-025-09761-x>
- [2] L. Baird. Residual algorithms: reinforcement learning with function approximation. *ICML*, 1995.
- [3] R. S. Sutton, H. R. Maei, D. Precup, S. Bhatnagar, D. Silver, C. Szepesvári, E. Wiewiora. Fast gradient-descent methods for temporal-difference learning with linear function approximation. *ICML*, 2009.
- [4] O. Nachum, B. Dai. Reinforcement learning via Fenchel–Rockafellar duality. arXiv:2001.01866, 2020.
- [5] N. Malkin, M. Jain, E. Bengio, C. Sun, Y. Bengio. Trajectory balance: improved credit assignment in GFlowNets. *NeurIPS*, 2022. arXiv:2201.13259.
- [6] L. Brunswic, Y. Li, Y. Xu, S. Jui, L. Ma. A theory of non-acyclic generative flow networks. *AAAI*, 2024. arXiv:2312.15246.
- [7] D. Tiapkin, N. Morozov, A. Naumov, D. Vetrov. Generative flow networks as entropy-regularized RL. *AISTATS*, 2024. arXiv:2310.12934.